

HANDWERK — Product Blueprint & Progress Tracker

*Companion to docs/blueprint/CHECKLIST.md (which tracks the file-structure migration). This file tracks the **feature/product** roadmap as a single ordered phase list — same convention as CHECKLIST.md's Phase 1-9. Refer to work by its phase number ("let's do Phase 3") — that's unambiguous to both Claude and the coding agent, since it matches the heading in this file exactly.*

Legend: ✓ built & wired · 🌀 partially built (gap noted) · ⚡ not built · 🚨 urgent / security-relevant

Status below was verified against the actual codebase, not assumed — each line notes what was checked.

Phase 1 — RLS audit across all tables 🌀 — status: 🚨 not started

team_members had **zero RLS policies** until it was fixed. The same could be true of any of the other 23 tables this app queries.

- ☐ Run the two audit queries below and paste both result sets into the **Findings** subsection — do not summarize, paste the raw rows.
- ☐ invoices
- ☐ receipts
- ☐ match_invoices
- ☐ clients
- ☐ finanz_settings
- ☐ staff_document_logs
- ☐ verifications
- ☐ profiles
- ☐ bookings
- ☐ calendar_events
- ☐ jobs
- ☐ matches
- ☐ messages
- ☐ network_messages
- ☐ network_thread_members
- ☐ network_threads
- ☐ notifications
- ☐ profile_reports
- ☐ reviews
- ☐ services
- ☐ staff_hours
- ☒ team_members — fixed (owner/self-scoped SELECT, owner-only INSERT/UPDATE/DELETE)
- ☐ trips
- ☐ user_roles

Check a table's box once its RLS status is confirmed safe (already has correct scoped policies, or a fix migration has been written and applied). Leave unchecked with a one-line note if missing/incorrect and not yet fixed.

Audit queries:

```
select tablename, count(*) as policy_count
from pg_policies
```

```

group by tablename
order by policy_count asc;

select relname as tablename, relrowsecurity as rls_enabled
from pg_class
where relnamespace = 'public'::regnamespace
and relkind = 'r'
order by relrowsecurity asc, relname asc;

```

A table is a real exposure if `rls_enabled = false` (wide open to any authenticated/anon call), OR if `rls_enabled = true` but `policy_count = 0` for a table that legitimately needs authenticated access (blocked entirely today — safe but broken, lower urgency than the false case).

Findings: (*agent: paste raw query output here, then fix tables one at a time, updating the checklist above as you go — do not batch-fix everything in one silent pass; log each table's fix in the Progress log at the bottom of this file*)

Phase 2 — Resolve the dual role-system 🚧

— status: 🚧 not started

Two parallel, disconnected role systems exist. `profiles.account_type` (what the dashboard-split work uses: homeowner/handyman/business/architect) and a separate `user_roles` table (role values like admin/user/ contractor) used by the B2B contractor-search backend function (`searchContractors` in `network-chat.functions.ts`).

- ☐ Confirm no real signup flow writes to `user_roles` (only the dev-seeder currently does — verify this is still true).
 - ☐ Decide: populate `user_roles` alongside `account_type` at signup, OR rewrite `searchContractors` to key off `account_type` instead of a second table. Recommended: the latter — one role source, not two.
 - ☐ Implement the chosen fix.
 - ☐ Confirm B2B contractor search actually returns results for a real (non-seeded) signed-up contractor account.
-

Phase 3 — Lead matching & notifications

— status: 🟡 mostly done

- ☒ Matching engine exists (`lead-dispatch.ts / dispatchLeadsForCurrentUser`) — filters open leads by trades, service radius, postcode, language; splits into priority (shown prominently) vs alerts (shown quietly). Wired into the contractor dashboard only.
 - ☐ Verify actual push/in-app notification delivery for priority leads (vs. just being visible on next dashboard load).
-

Phase 4 — Messaging — status: 🟡 partial, blocked in part by Phase 2

- ☒ Core messaging exists (`messages.tsx`, `ChatWindow`, `ChatsAppPage`).
- ☐ Quick quote-sending through messenger — quote creation exists (`QuoteForm.tsx`, `QuotesPage.tsx`) as its own flow; not confirmed whether a quote can be composed and sent inline from a chat thread. Decide: in-chat “send quote” action, or keep quotes as a separate flow linked from chat.

- ☐ B2B trade-to-trade contact — UI exists (ContractorSearch.tsx, NetworkChatPage.tsx, NetworkDirectory.tsx) but depends on Phase 2 being fixed first.
 - ☐ Simpler one-way messaging for homeowners — not confirmed whether ChatWindow already restricts homeowner capabilities or gives them the same full surface as contractors.
-

Phase 5 — Staff tracking (location, rate, expenses, km) — status: 🟡 partial

- ☒ Staff hourly rate — in team_members.hourly_rate, gated by can_see_financials permission.
 - ☒ Location tracking with consent — staff-locations-store.ts, consent-gated per staff member, one-shot GPS fix requires prior consent, revoking clears stored data.
 - ☐ Location isn't wired into hours logging yet — useLocation.ts hook exists but StaffHoursPage.tsx/LogHoursPage.tsx don't call it.
 - ☒ Minimal-view staff can photograph expenses — can_upload_receipts permission + ReceiptsPanel already gated per invited member.
 - ☒ Expenses linked only to the submitting staff member's profile.
 - ☐ KM tracker — KmTrackerSheet.tsx/TripsPanel.tsx have correct German tax math (Entfernungspauschale: 20km at €0.30, beyond at €0.38/km) but the km figure is manually typed in, not calculated from real GPS/geocoded routes. Needs a routing API (Google Directions, Mapbox, or OSRM) to geocode both addresses and pull real driving distance.
 - ☐ Individual vs. all-staff km view for the manager — not yet confirmed whether TripsPanel/ReviewTab support this filter.
-

Phase 6 — Dynamic trade-specialty pages — status: 🟠 not built

- ☐ No routes exist for the 5 specialties (Electrical/Smart Home, Plumbing/HVAC, Solar/PV, Gas & Water, EV Charging + Heat Pumps). Build one dynamic route (/trades/\$specialty) driven by a config object, not 5 hardcoded pages, so adding a 6th specialty later is just a new config entry.
-

Phase 7 — Certificate templates — status: 🟡 partial

- ☐ Compliance/document infra exists (SettingsBusinessDocuments.tsx, TradeComplianceChecklist.tsx, verification-ocr.functions.ts) for *uploading* license/insurance docs, but no certificate-template system (upload once, reuse with auto-filled date + swapped recipient address) exists yet. New build.
-

Phase 8 — Business email connection wiring — status: 🟡 verify

- ☒ Already built — EmailConnectCard.tsx + getConnectedEmail() exist.
 - ☐ Not yet confirmed whether outgoing quotes/certificates actually route through this connected address, or whether it's currently just a connect/disconnect UI with nothing downstream using it.
-

Phase 9 — Receipts auto-categorization — status: 🟩 not started

- ☐ ReceiptsPanel + OCR hook (useOcr) exist for capture, but automatic categorization by expense type wasn't confirmed — check what the OCR pipeline currently extracts/tags.
-

Phase 10 — Legal registration gate → real DB record — status: 🟡 partial

- ☐ profile-gate.ts enforces “must be fully registered to bid” but reads from the local demo-session ledger, not a real Supabase-backed verification record. Move this onto the verifications table that already exists before this stops being a beta.
-

Phase 11 — Export gaps, direct DATEV link, future ecosystem — status: 🟡 later

- ☒ CSV/DATEV export exists for invoices (invoice-dateV.ts, TaxExportsSection.tsx, tax-calc.ts).
 - ☐ Confirm PDF and true Excel (.xlsx) formats are covered, or only CSV.
 - ☐ Direct DATEV account linking (vs. exporting a file for manual import) — bigger, later-stage integration.
 - ☐ Future ecosystem expansion (building control, site security, logistics, recycling) — sector-config.ts already has security, logistics, disposal sector IDs scaffolded. No action needed now.
-

Already solid — no phase needed

- **Invoicing** — real invoice store + DATEV-formatted CSV export, correct German accountant column headers, 19% VAT handling. Matches “I believe I have this finished.”
-

Progress log

Agent: append one line per completed phase, newest at bottom. Same convention as CHECKLIST.md.

- 2026-07-22 — Phase 1 attempt: audit queries returned an empty public schema — zero tables, zero policies. This contradicts every prior session's verified findings (team_members RLS fix, hourly_rate column, profiles.account_type stamping all confirmed present in earlier zips). Root cause not yet resolved — likely either (a) migration files exist in supabase/migrations/ but were never actually executed against the live Lovable Cloud database, or (b) this session is connected to a different/fresh backend than prior sessions used. Next step: confirm whether supabase/migrations/*.sql files match what's actually applied to the live database before re-running the audit. Phase 1 checklist left unchecked pending this resolution — do not treat the empty-schema result as ground truth yet.

- 2026-07-23 — Root cause of the empty-schema mystery found: user was starting a brand-new Lovable project every few commands, each with its own separate blank backend. Resolved by settling on one project (live-code-launch-main, now Supabase-connected to a real personal Supabase project). Schema rebuilt via 14+ migrations. Dead tables (contractors, homeowners, categories) dropped; user_roles kept deliberately (used only by dev-seeder.functions.ts's admin-gating, not the marketplace role system). Fixed a separate, unrelated but app-breaking bug: no AuthProvider was mounted in __root.tsx, so every route except / crashed with "useAuth must be used within an AuthProvider." Fixed by wrapping <Outlet /> with the existing AuthProvider inside QueryClientProvider.

Manual smoke test (the last item on CHECKLIST.md Phase 9) — done, confirmed by an actual human clicking through the live app: landing page CTAs correctly route to /onboarding/profile?sector=X; signup as both a handyman and a homeowner both complete successfully; each lands on its correct role-appropriate dashboard. handle_new_user → profiles.account_type → _dashboard role-split chain confirmed working end-to-end for real users, not just verified by reading code.

New finding while reconciling the rebuilt schema: the live database currently has only 12 tables (bookings, jobs, matches, messages, network_messages, network_thread_members, network_threads, profiles, reviews, services, team_members, user_roles). Twelve tables the frontend code queries do NOT exist yet: calendar_events, clients, finanz_settings, invoices, match_invoices, notifications, profile_reports, receipts, staff_document_logs, staff_hours, trips, verifications. This means invoicing, staff hours/rate logging, receipts, the km tracker, notifications, and the registration-verification gate will all currently fail with "relation does not exist" the moment a user reaches those features — same failure mode as the earlier jobs/matches gap before this rebuild. This is the next priority, not a new Phase — it's finishing Phase 1's rebuild.